

# Principles of Bayesian statistics

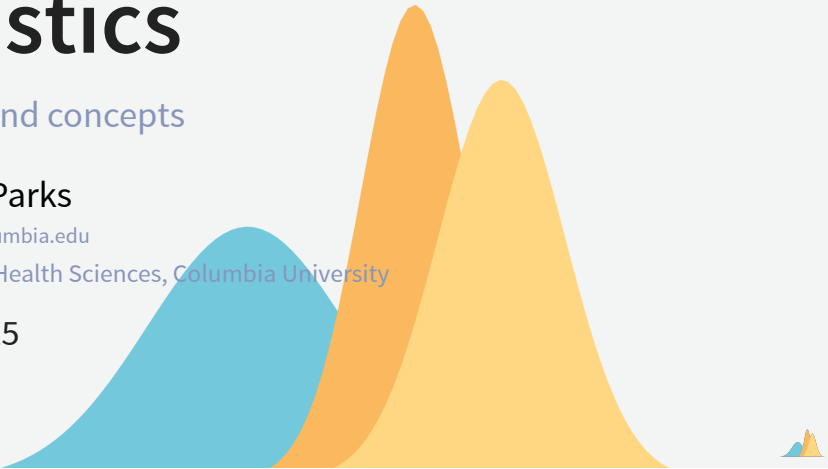
Key ideas and concepts

Robbie M. Parks

[robbie.parks@columbia.edu](mailto:robbie.parks@columbia.edu)

Environmental Health Sciences, Columbia University

Aug 14, 2025



## Outline

- Overview
- Introduction to Bayesian methods and concepts
- Using **NIMBLE** for Bayesian inference

## Overview

This course is for those who

# R

- **R** is an interactive environment developed by statisticians for data analysis.
- A more detailed Introduction to **R** can be found at <https://www.r-project.org>.
- **R** is the environment we will use throughout the workshop.
- But this isn't a course to learn R...
- There will lots of code, though we will restrict the equations as much as possible.

# R code

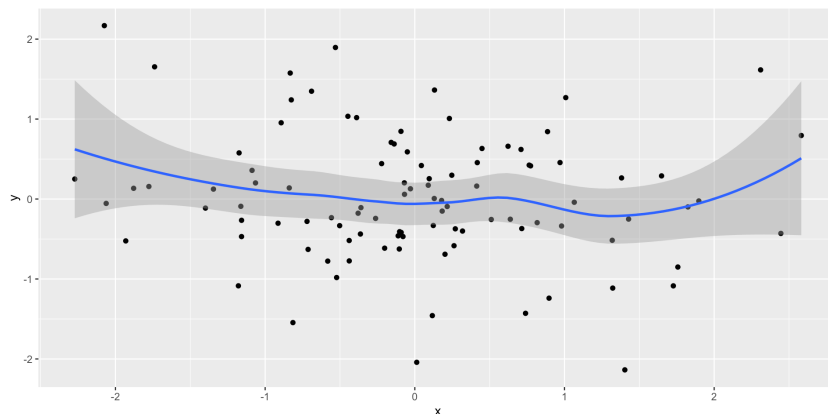
- Sample **R** code:

```
1 # Load packages
2 library(ggplot2)
3
4 # Create a dataset
5 set.seed(100)
6 data = data.frame(x=rnorm(100),y=rnorm(100))
7
8 # Plot and rough fit
9 p <- ggplot(data, aes(x, y)) +
10   geom_point() +
11   geom_smooth(method = "loess")
12
13 plot(p)
```



# R code output

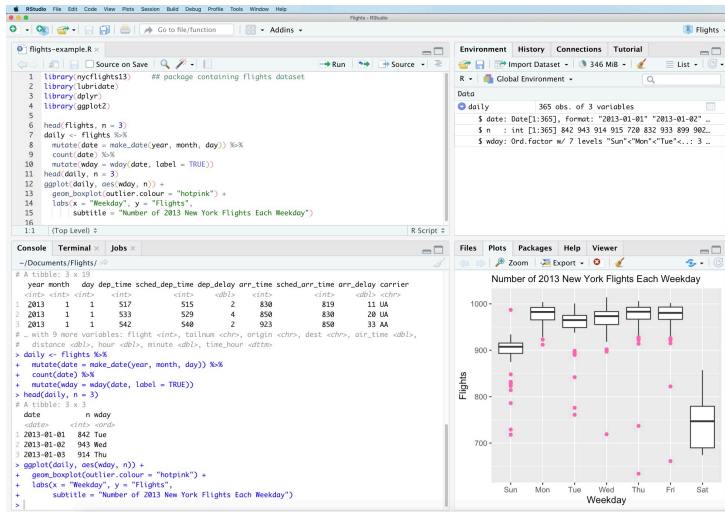
- Output of code:



# RStudio by Posit

- **RStudio** is an integrated development environment (IDE) for **R**.
- **RStudio** provides a convenient graphical interface to **R**, making it more user-friendly, and providing many useful features.
- Such features include direct code execution, tools for plotting, history, debugging, and workspace management.
- A more detailed background on to **RStudio** can be found at <https://posit.co/about/>.

# RStudio by Posit



# Posit (RStudio) Cloud

- **Posit Cloud** is a cloud-based **RStudio** which runs projects and code in the cloud.
- **Posit Cloud** allows convenient scaling and sharing of code, including for training programs such as SHARP.
- Registration for free is available via <https://posit.cloud/>.
- We will go through how to navigate **RStudio Cloud** together.
- More details are available via <https://docs.posit.co/cloud/>.



# GitHub

# NIMBLE

- **NIMBLE** is one of several Bayesian inference packages.
- **NIMBLE** code is written in a slightly unusual format if you're used to just using base **R**.
- Written in the style of a program called **BUGS**, developed at Imperial College London.
- But you don't really need to know about **BUGS** to be able to use **NIMBLE**.
- In the lab after this, we will start with some straightforward examples for situations you're likely familiar with to introduce the style of writing models.
- These examples will feature basic regression models using linear predictors.



## R with NIMBLE (from lab immediately after this)

- Example basic linear multi-predictor regression:

$$y_i \sim \text{Normal}(\mu_i, \sigma) \quad i = 1, \dots, N$$

$$\mu_i = \alpha + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$



## R with NIMBLE (from lab immediately after this)

- Sample NIMBLE code:

```
1 code <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dnorm(0, sd = 100) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) { # n is the number of observations we have in the data
11    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual
12    y[i] ~ dnorm(mu[i], sd = sigma)
13  }
14 })
```



## R with NIMBLE (from lab immediately after this)

- As written in frequentist form:

```
1 model_freq <- lm(
2   y ~ x1 + x2 + x3,
3   data = df
4 )
```



## R with NIMBLE (from lab immediately after this)

- Comparing frequentist code...

```
1 model_freq <- lm(
2   y ~ x1 + x2 + x3,
3   data = df
4 )
```

- ...with NIMBLE code:

```
1 code <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dnorm(0, sd = 100) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) { # n is the number of observations we have in the data
11    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual entry of linear predictors
12    y[i] ~ dnorm(mu[i], sd = sigma)
13  }
14 })
```



# Introduction to Bayesian methods and concepts

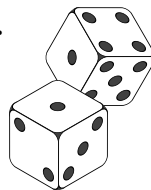
## Classical probability

- “Frequentist”.
- Limit of long-run relative frequency.
- Ratio of the number of times event occurred to the number of trials.



## Classical probability

- Consider rolling a fair six-sided die many times.
- We are seeing how often we roll a 2.
- We roll die a large number of times (N).
- Probability that we roll a 2:



- $\Pr(X = 2) = N(X = 2)/N$
- This will tend to  $1/6 = 0.16666666\dots$  as  $N \rightarrow \infty$

## Classical probability

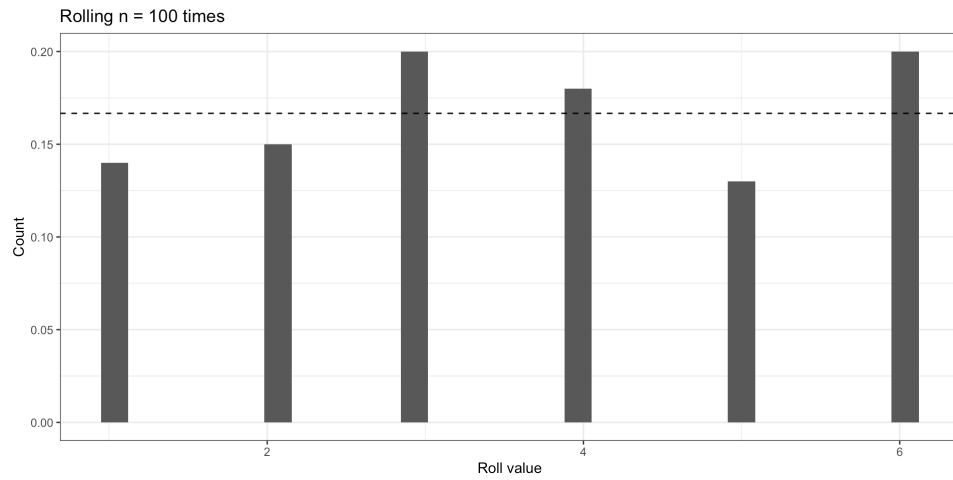
```

1 # Load packages
2 library(ggplot2)
3 library(purrr)
4
5 # Create dataset
6 set.seed(100)
7 n=100
8 data = data.frame(x=rdunif(n,6))
9
10 # Plot
11 ggplot(data) +
12   geom_histogram(aes(x,y=after_stat(count)/sum(after_stat(count)))) +
13   geom_hline(yintercept = 1/6, linetype=2) +
14   xlab('Roll value') + ylab('Count') +
15   ggtitle(paste0('Rolling n = ',n,' times')) +
16   theme_bw()

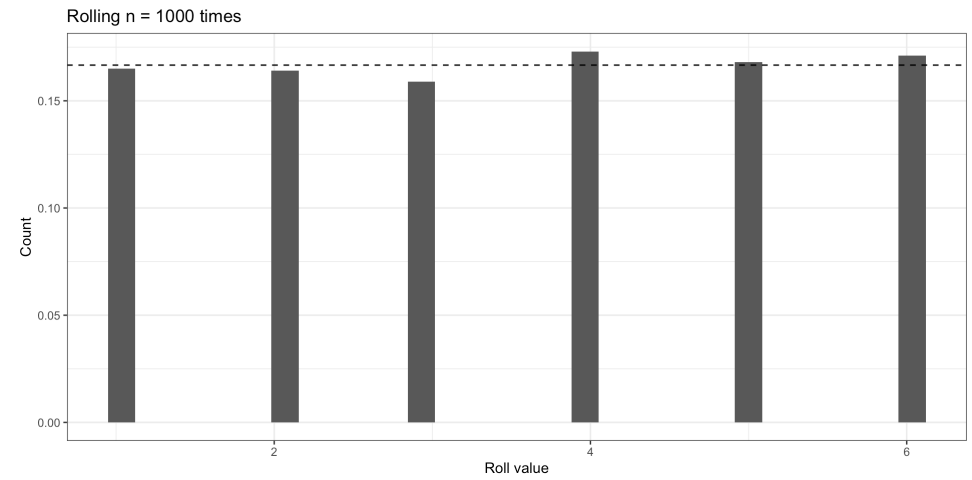
```



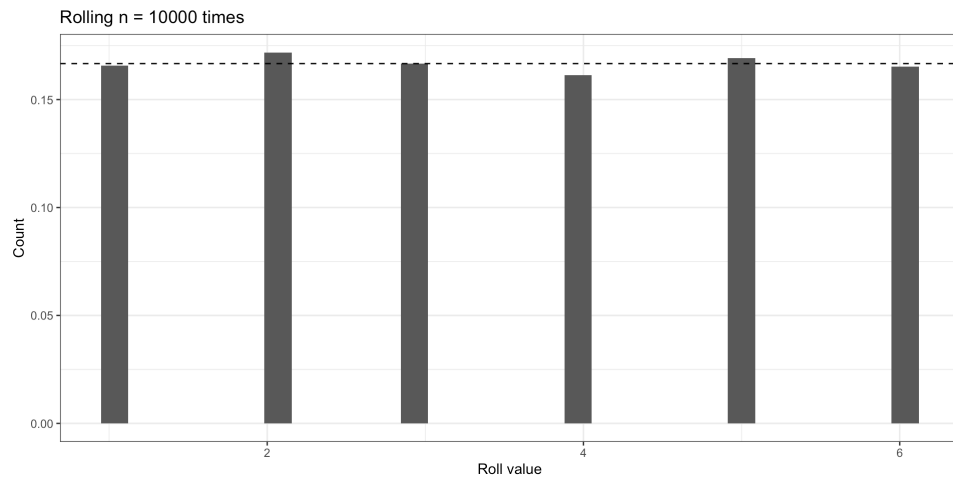
# Classical probability



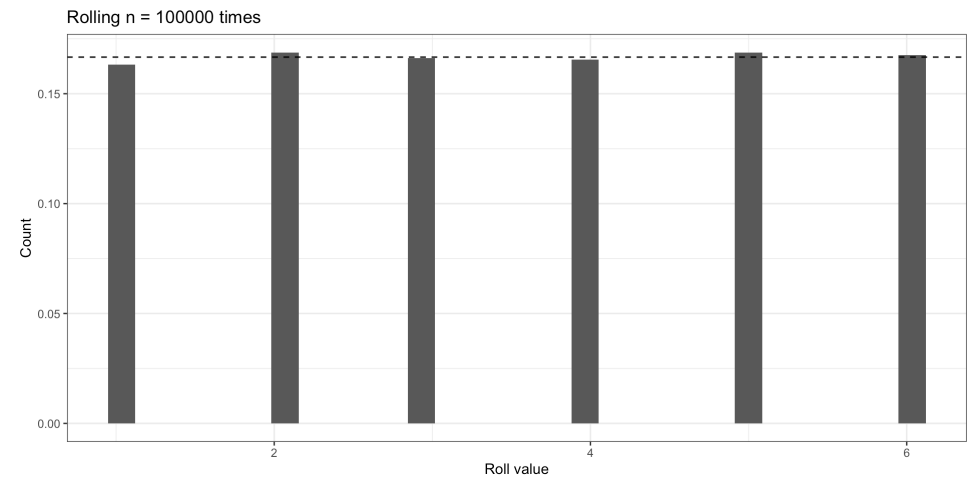
# Classical probability



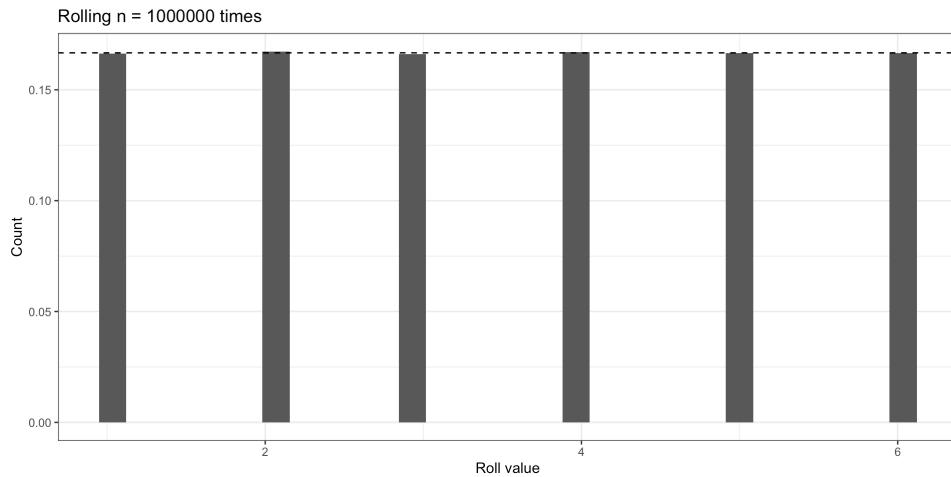
# Classical probability



# Classical probability



# Classical probability



# Subjective probability

- What is it?
  - Probability as degree of belief, not long-run frequency.
  - Quantifies uncertainty based on prior knowledge, expert opinion, or experience.
- Key Features
  - Personal: Two people can assign different probabilities to the same event — both valid
  - Coherent: Must follow the rules of probability (e.g., total = 1).
  - Updatable: Modified via Bayes' Theorem when new data arrives.
- Example
  - A researcher believes there's a 70% chance a new intervention reduces mortality, based on past studies and domain knowledge — even before seeing the data.

# Thomas Bayes and Simon Pierre Laplace

- Started with these two.
- Bayes (1701-1761) is why it's called **Bayesian**.
- Sorry Laplace (1749-1825)!



# Bayes theorem, prior, likelihood, and posterior

- $$P(\theta|y) \propto P(y|\theta)P(\theta)$$
- $P(\theta)$  is the **prior** (known or estimated a priori).
- $P(y|\theta)$  is the **likelihood** (probability of data given parameters of interest).
- $P(\theta|y)$  is the **posterior** (parameters of interest given data).

# Bayesian inference

## Choosing the prior distribution

- Prior choice can be vital.
- Priors reflect subjective probability.
- Type of distribution (we will see in a second).



## Conjugacy

- When the posterior is an analytical solution of the prior and likelihood.
- Examples include:
  - Normal data and Normal prior.
  - Binomial data and Beta prior.
  - Poisson data and Gamma prior.



## Normal-normal conjugacy example

Let:

- Observations:  $y_1, \dots, y_n \sim N(\theta, \sigma^2)$ , with known  $\sigma^2$
- Prior:  $\theta \sim N(\mu_0, \sigma_0^2)$
- Then the **posterior** is:

$$\theta \mid y \sim N(\mu_n, \sigma_n^2)$$

- where:

$$\sigma_n^2 = \left( \frac{n}{\sigma^2} + \frac{1}{\sigma_0^2} \right)^{-1}$$

$$\mu_n = \sigma_n^2 \left( \frac{n\bar{y}}{\sigma^2} + \frac{\mu_0}{\sigma_0^2} \right)$$





## Non-conjugacy (most of the time in real world)

- Because most prior-likelihood-posterior relationships are non-conjugate.
- No closed-form posterior (unlike normal-normal example).
- Need to approximate posterior from sampling methods.
- Need non-analytical solution to infer distributions.
- This is where ‘brute force’ sampling of distributions takes place.



36

## Obtaining approximate posterior distribution

## Obtaining approximate posterior distribution

- Many different types of MCMC:
  - **No-U-Turn Sampler (NUTS):**
    - An extension of HMC that automatically tunes parameters.
    - Avoids manual tuning of step size and trajectory length.
    - Used in **Stan** and **PyMC**
- Also non-MCMC:
  - **Variational Inference (VI):**
    - Optimizes a simpler distribution to approximate the posterior.
    - Much faster than MCMC, but gives an **approximation**, not exact samples.
    - Useful in large-scale models (e.g., deep learning).



37

## Sampling posterior

- Example basic linear multi-predictor regression:

$$y_i \sim \text{Normal}(\mu_i, \sigma) \quad i = 1, \dots, N$$

$$\mu_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3}$$

$$\alpha = 0$$

$$\beta_1 = 0.2$$

$$\beta_2 = 0.5$$

$$\beta_3 = 0.3$$

$$\sigma = 0.5$$

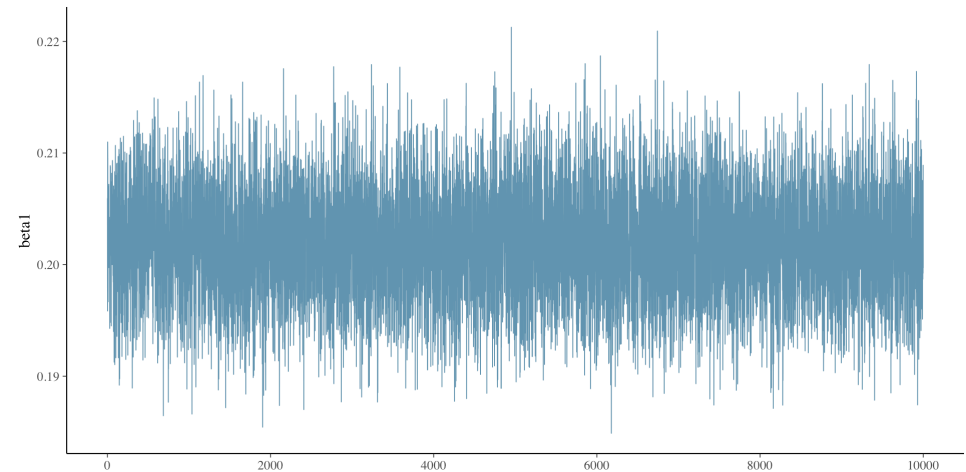


## Informative or non-informative priors?

- A source of intrigue from pre-Bayesians is how priors are chosen.
- Priors can be informed by existing knowledge.
- But what if we don't know anything really prior to inference?
- Most of the time we will not have analytical solutions.
- We will discuss choosing priors in more detail in Bayesian workflow lecture.



## Sampling posterior



## Example of influence of priors on sampling

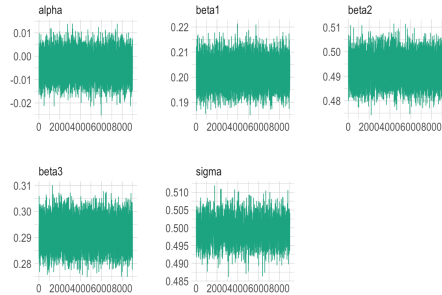
- Wide priors

```
1 code <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dnorm(0, sd = 100) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) { # n is the number of observations we have in the data
11    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual
12    y[i] ~ dnorm(mu[i], sd = sigma)
13  }
14 })
```



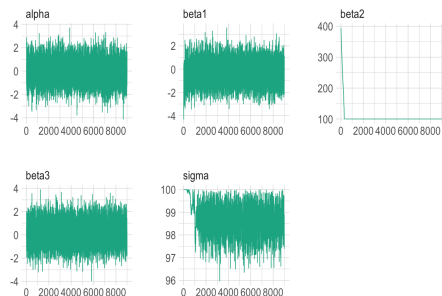
## Example of influence of priors on sampling

- Wide priors



## Example of influence of priors on sampling

- Worse priors



## Example of influence of priors on sampling

- Worse priors

```
1 code_worse_priors <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dunif(100, 100000) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) {
11    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual
12    y[i] ~ dnorm(mu[i], sd = sigma)
13  }
14 }
```



## How to get non-conjugate posterior?

- Some kind of software to help us implement models for inference.
- Using [R](#).
- Packages ([BUGS](#), [STAN](#), [INLA](#) etc.).
- We will be focusing on using [NIMBLE](#) throughout the two days.



# Using **NIMBLE** for Bayesian inference

## Illustrative example of basic regression in **NIMBLE**

- Let's go through an example from the upcoming lab.



## Linear regression

- Example basic linear multi-predictor regression:

$$y_i \sim \text{Normal}(\mu_i, \sigma) \quad i = 1, \dots, N$$

$$\mu_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3}$$

$$\alpha = 0$$

$$\beta_1 = 0.2$$

$$\beta_2 = 0.5$$

$$\beta_3 = 0.3$$



## Why we're using **NIMBLE** for (almost) everything during the workshop

- Interpretability.
- Flexibility.
- There are other Bayesian packages.
- You will see on Day 2...



# Linear regression

- First create some example data for our model:

```
1 set.seed(1)
2 p <- 3 # number of explanatory variables
3 n <- 10000 # number of observations
4 X <- matrix(round(rnorm(p * n), 2), nrow = n, ncol = p) # explanatory varia
5 true_betas <- c(0.2, 0.5, 0.3) # coefficients for beta1, beta2, beta3
6 sigma <- 0.5 # variance of variability around perfect agreement
7 y <- rnorm(n, X %*% true_betas, sigma)
```



# Linear regression

- What does equivalent frequentist model output look like for reference?

```
1 model_freq <- lm(
2   y ~ x1 + x2 + x3,
3   data = df
4 )
5 central_est <- t(t(model_freq$coefficients))
6 conf_int <- confint(model_freq)
7 cbind(central_est, conf_int)
```



# Linear regression

- What does the dataset look like?

```
# A tibble: 6 × 4
      y      x1      x2      x3
  <dbl> <dbl> <dbl> <dbl>
1 -0.145 -0.63 -0.8  0.24
2  0.0248 0.18 -1.06 0.24
3 -1.09 -0.84 -1.04 -0.64
4 -1.00  1.6 -1.19 -1.93
5 -0.138 0.33 -0.5  1.04
6  0.345 -0.82 -0.52 -0.28
```



# Linear regression

- What does equivalent frequentist model output look like for reference?

```
                2.5 %      97.5 %
(Intercept) -0.00272699 -0.01251412 0.007060142
x1           0.20196677  0.19229912 0.211634407
x2           0.49277663  0.48289869 0.502654568
x3           0.29160573  0.28189340 0.301318063
```



# Linear regression

- **NIMBLE** adopts and extends **BUGS** as a modeling language and lets you program with the models you create.
- Looks like below (as seen earlier in lecture and later in lab).

```
1 library(nimble)
2 code <- nimbleCode({
3   # priors for parameters
4   alpha ~ dnorm(0, sd = 100) # prior for alpha
5   beta1 ~ dnorm(0, sd = 100) # prior for beta1
6   beta2 ~ dnorm(0, sd = 100) # prior for beta2
7   beta3 ~ dnorm(0, sd = 100) # prior for beta3
8   sigma ~ dunif(0, 100) # prior for variance components
9
10  # regression formula
11  for (i in 1:n) { # n is the number of observations we have in the data
12    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual entry of linear predictors
13    y[i] ~ dnorm(mu[i], sd = sigma)
14  }
15 })
```



# Linear regression

- Run the MCMC simulations. There are lots of arguments in the main function, nimbleMCMC().

```
1 tic <- Sys.time()
2 nimbleMCMC_samples_initial <- nimbleMCMC(
3   code = code,
4   data = data,
5   constants = constants,
6   inits = inits,
7   niter = 10000, # run 10000 samples
8   setSeed = 1,
9   samplesAsCodaMCMC = TRUE
10 )
11
12 toc <- Sys.time()
13 toc - tic
```



# Linear regression

- Setting up data (will explain more in lab)

```
1 x1 <- X[, 1] - mean(X[, 1])
2 x2 <- X[, 2] - mean(X[, 2])
3 x3 <- X[, 3] - mean(X[, 3])
4
5 constants <- list(n = n)
6 data <- list(y = y, x1 = x1, x2 = x2, x3 = x3)
7
8 inits <- list(alpha = 0, beta1 = 0, beta2 = 0, beta3 = 0, sigma = 1)
```



# Linear regression

- Run the MCMC simulations. There are lots of arguments in the main function, nimbleMCMC().

```
|-----|-----|-----|-----|
|-----|-----|-----|-----|
Time difference of 50.31507 secs
```



## Linear regression

- The following operations will help us to answer: What is the summary of each estimated parameter from the samples?

```
1 library(bayesplot)
2 library(posterior)
3 library(hrbthemes)
4 summarise_draws(nimbleMCMC_samples_initial, default_summary_measures())
```



## Linear regression

- Then examine how well the model has converged, which typically is identified by how close each rhat value is to 1.00.

```
1 summarise_draws(nimbleMCMC_samples_initial, default_convergence_measures())
```



## Linear regression

- The following operations will help us to answer: What is the summary of each estimated parameter from the samples?

```
# A tibble: 5 × 7
  variable    mean  median    sd    mad    q5    q95
  <chr>      <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
1 alpha   -0.00395 -0.00388 0.00500 0.00498 -0.0122 0.00426
2 beta1    0.202    0.202  0.00492 0.00485  0.194  0.210
3 beta2    0.493    0.493  0.00506 0.00511  0.484  0.501
4 beta3    0.292    0.292  0.00499 0.00502  0.283  0.300
5 sigma    0.499    0.499  0.0122  0.00372  0.493  0.505
```



## Linear regression

- Then examine how well the model has converged, which typically is identified by how close each rhat value is to 1.00.

```
# A tibble: 5 × 4
  variable  rhat  ess_bulk  ess_tail
  <chr>    <dbl>    <dbl>    <dbl>
1 alpha   1.000    9936.    9714.
2 beta1   1.000   10026.   10110.
3 beta2   1.000    9990.    9857.
4 beta3   1.000    9536.    9755.
5 sigma   1.00     665.     190.
```



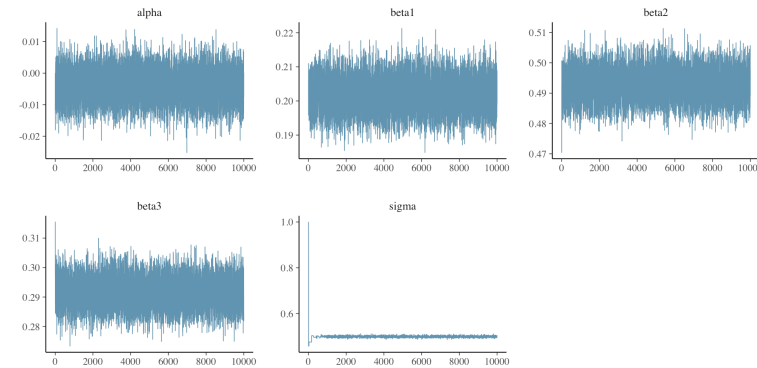
# Linear regression

- What do the samples of one of the unknown parameters actually look like?

```
1 mcmc_trace(nimbleMCMC_samples_initial)
```

# Linear regression

- What do the samples of one of the unknown parameters actually look like?



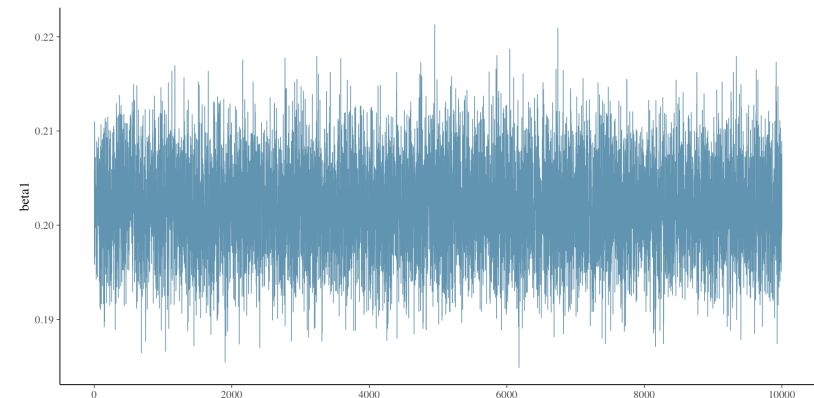
# Linear regression

Let's now focus on **beta1** (which we know is 0.2 from setting up initially).

```
1 mcmc_trace(nimbleMCMC_samples_initial, pars = c("beta1"))
```

# Linear regression

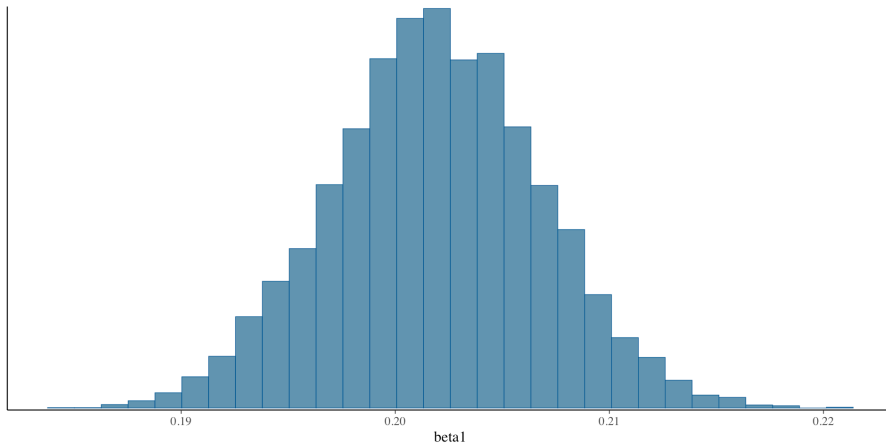
Let's now focus on **beta1** (which we know is 0.2 from setting up initially).





# Linear regression

Plotting `beta1` samples (posterior) as a histogram...



## Getting ready for the lab



## The lab for this session

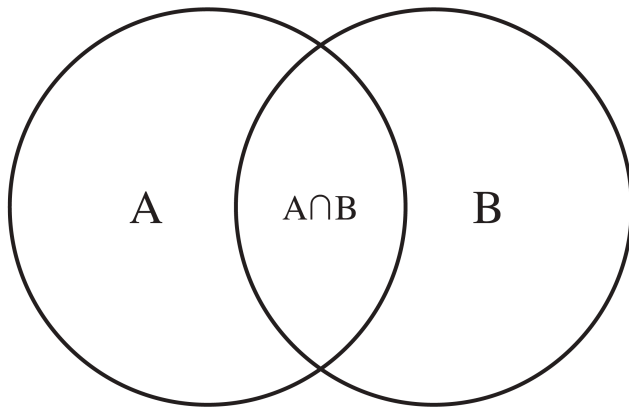
- This lab will involve taking some models and concepts from the Introduction to Bayesian Methods lecture and introduce you to the way `NIMBLE` works.
- During this lab session, we will:
  1. Explore how `NIMBLE` is written and works;
  2. Write some common regression models (Normal, logistic regression, Poisson);
  3. Understand how basic model assessment is made; and
  4. Test out how adjustments of models are made via different priors and more samples.

## Questions?



# Extra

## Conditional probability



## Conditional probability

- Two events A and B.
- Conditional event A given B.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$



## Conditional probability

In a group of 100 SHARP workshop attendees, 40 chose to stay in Manhattan, 30 chose to stay for the entire week, and 20 chose to stay in Manhattan and for the entire week. If a car buyer chosen at random chose to stay in Manhattan, what is the probability they chose to stay for the entire week?

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = 20/40 = 0.5$$



## Conditional probability

- $P(A|B) = \frac{P(A \cap B)}{P(B)}$
- can also be written as...
- $P(A \cap B) = P(A|B)P(B)$
- Also...
- $P(B|A) = \frac{P(A \cap B)}{P(A)}$
- Substituting top into bottom gets...
- $P(B|A) = \frac{P(A|B)P(B)}{P(A)}$



## Example of a conjugate pair: Normal-Normal

- Posterior:
- Because of conjugacy, there is an analytical solution for the posterior which is also normally distributed.
- But this is usually not that useful when models get very complicated.



## Example of a conjugate pair: Normal-Normal

- Likelihood:
- Normal distribution is defined by two parameters  $\mu$  and  $\sigma$ .
- Bell curve.
- $\mu$  is mean.
- $\sigma$  is standard deviation.
- $y|\mu, \sigma \sim \text{Normal}(\mu, \sigma^2)$
- Prior:
- Normal distribution is mathematically the conjugate of itself, with hyperparameters  $\mu_0$  and  $\sigma_0$ .

- $\mu \sim \text{Normal}(\mu_0, \sigma_0^2)$



## Examples of Normal distributions

Normal(1, 0)



# Examples of Normal distributions

Normal(1, 0)



# Examples of Normal distributions

Normal(0, 20)



# Examples of Normal distributions

Normal(0, 5)



## Example of a conjugate pair: Binomial-Beta

- Likelihood:
- Binomial distribution is when there is success (1) or failure (0) with the proportion of success ( $\pi$ ).
- $n$  trials.

- $y|\pi \sim \text{Binomial}(\pi, n)$

- Prior:
- Beta distribution is mathematically the conjugate of binomial defined by two parameters  $a$  and  $b$ .
- Beta distribution is 'natural' fit because it ranges from 0 to 1.

- $\pi \sim \text{Beta}(a, b)$



## Example of a conjugate pair: Binomial-Beta

- Posterior:
- Because of conjugacy, there is an analytical solution for the posterior.
- $p(\pi|y) \sim \text{Beta}(y + a, n - y + b)$

## Examples of Beta distributions

Beta(2, 10)



## Examples of Beta distributions

Beta(0.5, 0.5)



## Examples of Beta distributions

Beta(5, 1)

# Examples of Beta distributions

Beta(5, 5)



# Examples of Beta distributions

Beta(50, 200)



# Examples of Beta distributions

Beta(5, 20)



## Example of a conjugate pair: Poisson-Gamma

- Likelihood:
- Normal distribution is defined by on parameters  $\lambda$ .
- Very common for count data.
- $\lambda$  is rate.

- $y|\lambda \sim \text{Poisson}(\lambda)$

- Prior:
- Gamma distribution is mathematically the conjugate of Poisson, with hyperparameters  $\mu_0$  and  $\sigma_0$ .
- Gamma distributuon is 'natural' fit becasue it ranges from 0 to  $\infty$ .

- $\lambda \sim \text{Gamma}(a, b)$



## Example of a conjugate pair: Poisson-Gamma

- Posterior:
- Because of conjugacy, there is an analytical solution for the posterior.
- $q|y \sim \text{Gamma}(a + y, b + E)$



## Examples of Gamma distributions

Gamma(1, 1)



## Examples of Gamma distributions

Gamma(0.1, 0.1)



## Examples of Gamma distributions

Gamma(3, 3)



# Examples of Gamma distributions

Gamma(3, 0.5)

